МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ» (ДГТУ)

Факультет
Информатика и вычислительная техника

Кафедра
Программное обеспечение вычислительной техники и автоматизированных систем

ВВЕДЕНИЕ В ЯЗЫК ПРОГРАММИРОВАНИЯ C++. ЗНАКОМСТВО С БАЗОВЫМИ АЛГОРИТМИЧЕСКИМИ КОНСТРУКЦИЯМИ И СТАТИЧЕСКИМИ МАССИВАМИ

Практикум

по выполнению лабораторной работы №1
по дисциплине «Программирование»

Программное обеспечение вычислительной техники и автоматизированных систем
(указывается направленность (профиль) образовательной программы)

02.03.03 Математическое обеспечение и администрирование информационных систем
(указывается код и наименование направления подготовки)

Ростов-на-Дону
2025 г.

Составители: канд. техн. наук, доц. каф. А.А. Скляренко, ст. преп. Атаян А.М.

УДК 004.432

Введение в язык программирования C++. Знакомство с базовыми конструкциями: метод. указания. – Ростов н/Д: Издательский центр ДГТУ, 2025.

В методическом указании рассматриваются вопросы создания консольных приложений на языке C++, консольный ввод/вывод, работа с базовыми типами данных и основными управляющими конструкциями языка. Приведены задания к лабораторной работе, помогающие закрепить на практике полученные знания, и контрольные вопросы для самопроверки.
Предназначено для обучающихся по направлению 02.03.03 «Математическое обеспечение и администрирование информационных систем», профиль «Большие данные и машинное обучение».

Ответственный за выпуск:
зав. кафедрой «Программное обеспечение вычислительной техники и автоматизированных систем» Долгов В.В.

© А.А. Скляренко,
А.М. Атаян, 2025
© Издательский центр ДГТУ, 2025

1 Теоретическая часть

1.1 Создание консольных приложений

Первое, что необходимо сделать, приступая к работе со средой Microsoft Visual Studio –создать новый проект консольного приложения. Для этого в среде разработки в меню «File» выберите пункт «New/Project…» (рисунок 1).

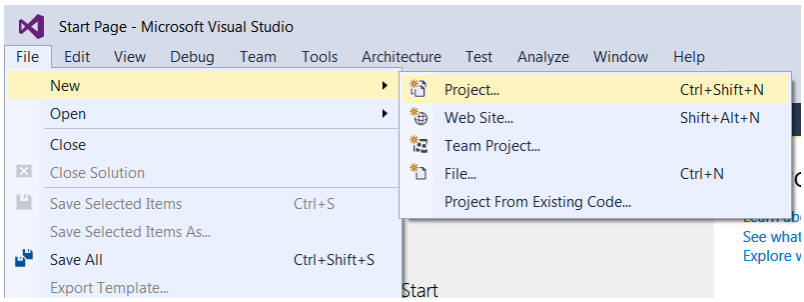


Рисунок 1 – Создание проекта через главное меню Visual Studio

Откроется диалоговое окно создания нового проекта (рисунок 2). В левом списке из всех возможных типов доступных проектов выберите язык «Visual C++» и тип платформы «Win32» (рисунок 2, область 1). В центральной области диалогового окна выберите предустановленный шаблон проекта «Win32 Console Application» (рисунок 2, область 2), а в поле «Name» (рисунок 2, область 3) введите имя создаваемого проекта. Опционально в поле «Location» диалогового окна (рисунок 2, область 4) можно также указать месторасположение файлов создаваемого проекта (например, специально отведенный для студентов сетевой диск или переносной накопитель). Учтите, что система проверяет отсутствие проекта с введенным именем и в случае совпадения имен сообщит об ошибке.

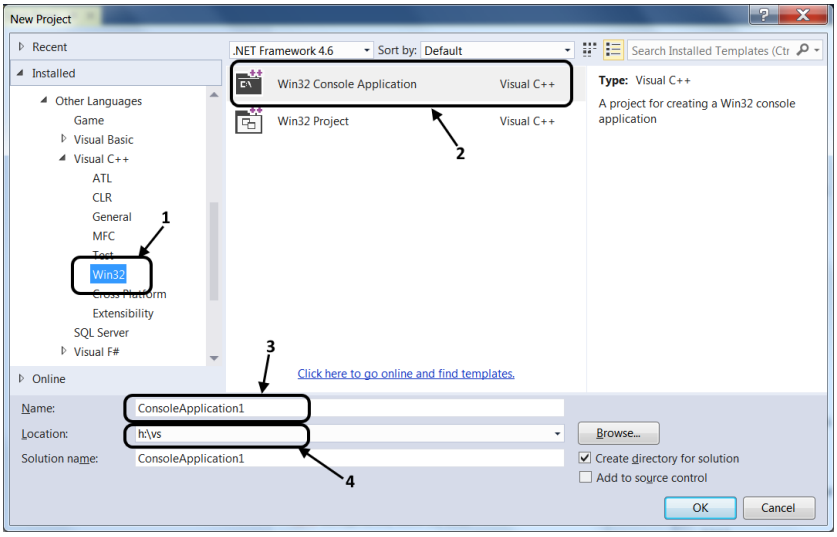


Рисунок 2 – Выбор типа проекта, его имени и расположения

После нажатия кнопки «ОК» отобразится окно мастера с набором настроек создаваемого проекта. Пропустите первую страницу мастера, а на второй установите настройки так, как это изображено на рисунке 3.

При успешности всех действий вновь созданный проект откроется в главном окне. В левой части среды разработки обычно расположена специальная область «Solution Explorer» отображающая структуру проекта и входящие в него файлы. Остальную область, как правило, занимает текстовый редактор, редактор оконных форм, страниц интернет-сайтов и другие специфичные для каждого конкретного случая области редактирования.

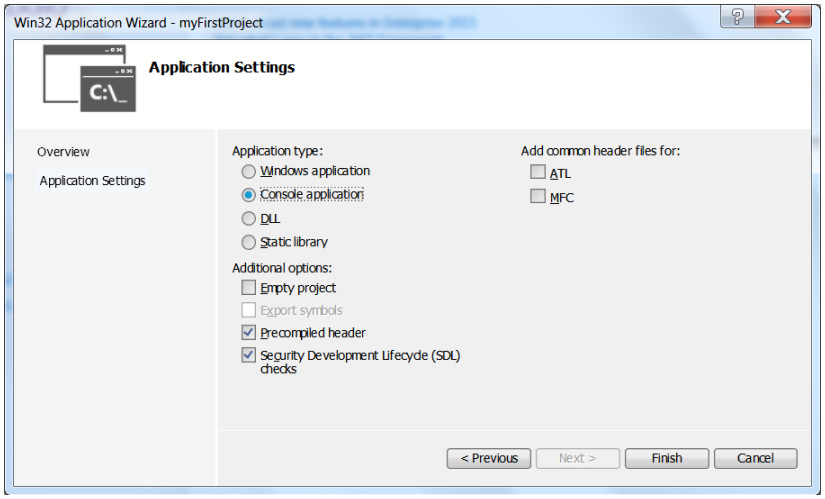


Рисунок 3 – Выбор типа приложения и используемых встроенных библиотек

Для проверки работоспособности среды к выполнению программы преобразуйте открытый в редакторе текст файла к виду, изображенному на рисунке 4.

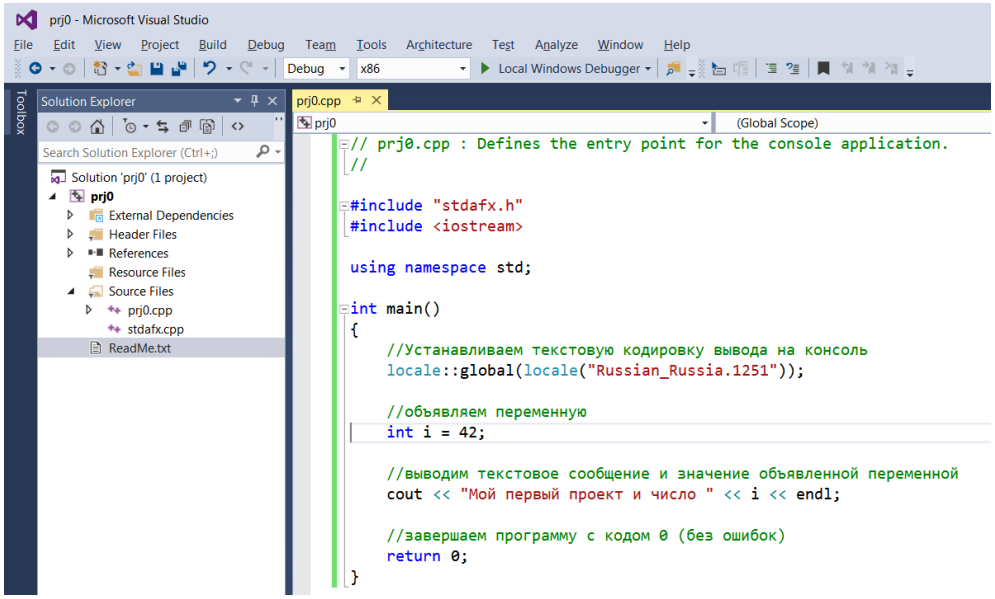


Рисунок 4 – Вид первого созданного в Visual Studio проекта с выводом текста

Часто, программы на самых различных языках, содержат комментарии, поясняющие код программы. В языке C++ существует два вида комментариев:

однострочные, начинающиеся символами «//» и продолжающиеся до конца строки программы (как в коде программы на рисунке 4);

многострочные, начинающиеся с символов «/*» и заканчивающиеся символами «*/».

Первые строки приведенной в примере программы содержат так называемые директивы препроцессора «#include», сообщающие компилятору о необходимости подключить заголовочные файлы «stdafx.h» и «iostream», в которых описаны стандартные функции языка, в том числе – команды ввода-вывода. В дальнейшем, в пособии, мы будет ссылаться на различные заголовочные файлы с теми или иными функциями, или типами данных. Когда вам необходимо будет использовать эти функции, вы также сможете подключить соответствующие файлы с помощью директивы «#include».

Далее, после директив препроцессора, расположено описание функции «main». Все программы на языке C++ обязательно содержат функцию «main», так как с нее начинается выполнение программы.

1.2 Основные программные конструкции языка C++

Изучение любого языка программирования всегда начинается с некоторых базовых элементов, заложенных в язык его разработчиками. В данной главе мы рассмотрим основные базовые типы данных встроенные в язык C++, операции, которые можно совершать над этими данными, и операторы, являющиеся основой любого алгоритма.

1.2.1 Типы данных, переменные и операции над ними

Язык C++ относится к языкам со строгой статической типизацией. Это означает, что каждая переменная, то есть именованная область памяти, хранящая информацию, имеет строго определенный тип данных, определяющий внутреннюю структуру хранения и операции, которые с этой информацией можно совершать.

Большинство программ на C++ будут основаны на наборе стандартных типов данных, большинство из которых перечислены в таблице 1.

Таблица 1 – Основные стандартные типы данных

Тип	Хранимые значения
char	Целочисленное значение в диапазоне от -128 до 127. Обычно используется для хранения текстовых символов. Занимает в памяти один байт.
int	Целочисленное значение в диапазоне от -2^{31} до $2^{31}-1$. Занимает в памяти 4 байта памяти.
unsigned	Целочисленное значение в диапазоне от 0 до $2^{32}-1$. Представляет собой беззнаковый вариант типа int. В памяти также занимает 4 байта.
long	Целочисленное значение, занимаемый размер в памяти и диапазон значений которого зависят от используемого компилятора. Для Visual Studio 2015 это соответственно 4 байта и диапазон от -2^{31} до $2^{31}-1$. Для компилятора Cross C++ (среды Eclipse) в LinuxMint x64 – 8 байт и диапазон от -2^{63} до $2^{63}-1$.
float	Знаковое число с плавающей точкой (нецелое). Диапазон значений абсолютной величины (модуля) которого может быть от $3.4*10^{-38}$ до $3.4*10^{38}$. Занимает в памяти 4 байта.
double	Знаковое число с плавающей точкой (нецелое). Диапазон значений абсолютной величины (модуля) которого может быть от $1.7*10^{-308}$ до $1.7*10^{308}$. Занимает в памяти 8 байта.

Прежде чем использовать переменную, ее нужно объявить, то есть указать тип данных этой переменной, ее имя, по которому программа будет обращаться к ней в дальнейшем, и необязательное инициализирующее выражение, значение которого будет присвоено объявляемой переменной.

<тип-данных> {<имя-переменной> [= <выражение>]};

Каждая переменная должна иметь уникальное имя (идентификатор), к которому предъявляются следующие требования:

В именах допускаются только латинские буквы, цифры и знак «_» (нижнее подчеркивание).

Имя не может начинаться с цифры.

Имя не должно содержать пробелов. Если нужно имя из нескольких слов, их чаще всего разделяют знаками подчеркивания «_» (some_variable_name) или начинают каждое из них с большой буквы (someVariableName).

Прописные и строчные буквы трактуются как разные. То есть, идентификатор «SUM» это не то же самое, что «sum» или «Sum».

Нельзя использовать в качестве имен ключевые слова, то есть те слова, которые в C++ трактуются как команды и операторы.

В современных версиях языка C++ предусмотрена возможность объявлять переменную, не указывая тип напрямую. При этом ее тип выводится из выражения инициализации, которое при таком объявлении становится обязательным. Для этого используется следующая синтаксическая конструкция:

auto <имя-переменной> = <выражение>;

Ключевое слово «*auto*» рекомендуется использовать в большинстве ситуаций, поскольку оно обладает рядом преимуществ:

- Стабильность кода. Если тип инициализирующего выражения изменился (в том числе, если изменен возвращаемый тип функции, см. пункт 4.1), это не повлияет на его работу.
- Эффективность. Тип переменной будет строго соответствовать инициализирующему выражению, никаких преобразований выполняться не будет.
- Удобство использования. Можно не беспокоиться об опечатках и ошибках в имени типа данных. Особенно это касается типов с длинными «говорящими» именами.

Примеры объявления переменных приведены ниже в листинге 1.

Листинг 1 – Примеры объявления переменных

```
int i, x;          //Объявление двух целых переменных
float y;           //Объявление вещественной переменной
int a = 5;         //Объявление целой переменной и ее
                   //инициализация значением 5
//автоматический вывод типа int из числа 10
auto myAuto = 10;
//автоматический вывод типа double из числа 4.2
auto x = 4.2;
```

В языке реализована поддержка символов и строковых констант (литералов) в формате UTF-8, UTF-16 и UTF-32. Для этого необходимо использовать символьные типы данных: char16_t и char32_t. При этом для интерпретации символьных литералов в соответствующей кодировке, им должны предшествовать префиксы «*u8*» (UTF-8), «*u*» (UTF-16) или «*U*» (UTF-32). Например, константное значение «U'a'» будет интерпретировано компилятором как символ в кодировке UTF-32.

1.2.2 Операции над данными, приоритет операций

Выражение – это конструкция языка, предназначенная для получения некоторого значения. Тип вычисленного значения определяет тип выражения. Выражение может содержать константы, переменные, знаки операций, вызовы функций, возвращающих значения и круглые скобки. Выражение не обязательно должно содержать все вышеперечисленные конструкции, например, одна константа или одна переменная являются частными случаями выражений.

Операции являются элементами выражений и также возвращают значение. Помимо собственно действия выполняемого операцией по вычислению значения, с каждой операцией связаны следующие атрибуты:

- арность (количество аргументов необходимых для ее выполнения);
- знак операции (символ или набор символов, которыми записывается операция в языке);
- приоритет;
- ассоциативность.

В таблице 2 представлены все стандартные операции C++, их приоритет (Р), ассоциативность (Асс.) и арность (Арн.).

Таблица 2 – Приоритеты и ассоциативность операций языка C++

Р	Операции	Асс.	Арн.	Примечание
15	`++`	→	1	Постинкремент.
	`--`		1	Постдекремент.
	`[]`		1	Обращение к элементу по индексу.
	`()`		1	Вызов функции.
	`.`		2	Доступ к полю составной структуры.
	`->`		2	Доступ к полю структуры через указатель.
14	`sizeof`	←	1	Получение размера выражения или типа данных в байтах.
	`++`			Прединкремент.
	`--`			Преддекремент.
	`&`			Получение адреса.
	`*`			Разадресация.
	`+ или -`			Знак выражения.
	`~`			Побитовое отрицание.

P	Операции	Acc.	Арн.	Примечание
	!			Логическое отрицание.
	(тип-данных)			Приведение типа данных.
13	*	→	2	Умножение.
	/			Деление.
	%			Остаток от целочисленного деления
12	+	→	2	Сложение.
	-			Вычитание.
11	>>	→	2	Побитовый сдвиг вправо.
	<<			Побитовый сдвиг влево.
10	<	→	2	Меньше (истина, если левый аргумент меньше правого).
	<=			Меньше или равно.
	>			Больше.
	>=			Больше или равно.
9	==	→	2	Проверка на равенство.
	!=			Проверка на неравенство.
8	&	→	2	Побитовая конъюнкция (И, AND).
7	^	→	2	Побитовое исключающее ИЛИ (XOR).
6	|	→	2	Побитовая дизъюнкция (ИЛИ, OR).
5	&&	→	2	Логическая конъюнкция (логическое И).
4	||	→	2	Логическая дизъюнкция (логическое ИЛИ).
3	?:	←	3	Тернарная операция условия.
2	=	←	2	Операция присваивания.
	+= -= *= /= %= &= |= ^= <<= >>=			Операция присваивания, совмещенная с предварительным выполнением другой указанной операции.
1	,	→	2	Операция запятая (операция последовательного выполнения).

Листинг 2 – Пример использования операций присваивания

```
#include <iostream.h>
int main(){
     int a=2, b=6, c=7;
     a=b; b*=2; c+=1;
     return 0;
}
```

Тернарная условная операция

Данная операция является в языке C++ единственной, у которой присутствуют три аргумента. Она имеет следующий синтаксис:

(условие) ? (выражение-да) : (выражение-нет)

Если условие операции истинно, то операция возвращает значение «выражение-да», иначе – значение «выражение-нет».

Например, при выполнении программы из листинга 3 переменная «ans» примет значение 10.

Листинг 3 – Пример использования тернарной операции

```
#include <iostream>
int main(){
     int a=4, b=3, ans;
     ans = (a > b) ? 10 : 25;
     return 0;
}
```

Операции инкремента и декремента

Инкремент (++) увеличивает, а декремент (--) уменьшает значение переменной на единицу. Данные операции могут быть представлены в двух видах: префиксном и постфиксном.

Префиксная форма операций выполняется перед тем как переменная, к которой она применена будет использована в вычислении остальной части выражения, а постфиксная – после того как все выражение будет вычислено (см. листинг 4).

Листинг 4 – Пример использования операций инкремента и декремент

```
#include <iostream>
 int main(){
     int value = 2015;

     //операция преинкремента,
     //переменные «x» и «value» примут значение 2016
     int x = ++value;

     //операция постинкремента,
     //переменная «y» примет значение 2016
     //a «value» - 2017.
     int y = value++;

     //операция предекремента,
     //переменные «z» и «value» примут значение 2016
     int z = --value;

     //операция постдекремента,
     //переменная «a» – 2016, a «value» – 2015
     int a = value--;
```

```
    return 0;
}
```

Операции сравнения

Операции сравнения (операторы отношений) сравнивают два выражения между собой. Результатом этих операция всегда является логическое значение – true или false (истина или ложь) в зависимости от значений сравниваемых выражений (см. листинг 7).

Листинг 7 – Пример использования операций сравнения

```
bool res;
int x, y;
res = (x == y);     //true, если x равно y, иначе false
res = (x == x);     //всегда true
res = (2 < 1); //всегда false
res = (x == 2);     //true, если x равно 2
res = (x > 5); //true, если x больше 5
res = (x <= 2);     //true, если x меньше или равно 2
res = (x != 2);     //true, если x не равен 2
```

Операции сравнения имеют более низкий приоритет, чем арифметические операции, поэтому выражение вроде «*i* < *lim-1*» будет выполняться так же, как и выражение «*i* < *(lim-1)*».

Если нужно составить сложное условие, состоящее из нескольких простых, то используются специальные логические операции, рассмотренные ниже.

Логические операции

В языке C++ существует три логические операции:

логическая операция И (&&);

логическая операция ИЛИ (||);

логическая операция НЕ (!) или логическое отрицание.

При выполнении этих операций любое значение выражения отличное от нуля будет рассматриваться как Истина, а значение равное нулю – как Ложь.

Логические операции помогают образовывать сложные (составные) условия из нескольких более простых. Например, как показано в листинге 8.

Листинг 8 – Пример использования логических операций

```
#include <iostream>
int main(){
    //объявление переменных
    bool a1 = true, a2 = false;
    bool a3 = true, a4 = false;
    int x = 3, y = 4;
```

```
    //логическое И
    bool b1 = a1 && a2;       //b1 = 0(false)
    bool b2 =  a1 && a3;           //b2 = 1(true)
    bool b3 = (x == 3) && (y != 4);    //b3 = 0(false)

    //логическое ИЛИ
    b1 =  a1 || a2;           //b1 = 1
    b2 =  a1 || a3;           //b2 = 1
    bool b3 = (x == 3) || (y != 4);    //b3 = 1(true)

    return 0;
}
```

Во многих выражения необходимо вычислить не просто произведение или сумму каких-либо значений, а произвести более сложные действия. И хотя на языке C++ можно самостоятельно описать любые известные алгоритмы, стандартные библиотеки языка уже предоставляют нам большой набор готовых функций.

Для использования таких функций в области математики необходимо подключить к программе заголовочный файл «math.h». Описание наиболее часто используемых функций из этого файла приведено в таблице 3.

Таблица 3 – Основные математические функции

Функция	Описание	Пример использования
abs(x)	Модуль числа	abs(-2.71) //2.71
ceil(x)	Округление вверх	ceil(2.5) //3 ceil(-2.5) //-2
cos(x)	Косинус (угол задается в радианах)	cos(0) //1
floor(x)	Округление вниз	floor(2.5) //2 floor(-2.5) //-3
fmod(x,y)	Остаток от деления x на y	fmod(3, 2) //1
pow(x,y)	Возведение в степень x^y	Pow(2, 3) //8
sin(x)	Синус (угол задается в радианах)	sin(0) //0
sqrt(x)	Квадратный корень числа	sqrt(9) //3
tan(x)	Тангенс (угол задается в радианах)	tan(0) //0

1.2.3 Организация ввода/вывода данных на консоль

В предыдущих пунктах были приведены примеры присвоения значений переменных непосредственно в теле программы, но они также могут быть введены с клавиатуры, а полученные значения выведены на консоль. Для этого используются специальные потоки вывода «cout» и ввода «cin», описания которых содержатся в заголовочном файле «iostream».

Также в языке можно использовать функции, оставшиеся по наследству от языка C (Си). Это функция вывода «printf» и функция чтения данных с консоли «scanf», определенные в библиотеке «stdio.h».

Рассмотрим простой пример программы вычисляющей площадь прямоугольника по его длине и ширине. Длина и ширина должны вводиться пользователем с клавиатуры, а площадь выводиться на консоль (см. листинг 10).

Листинг 10 – Код программы вычисления площади прямоугольника

```
#include <iostream>
using namespace std;
int main() {
	//переключим консоль на кирилический шрифт
	locale::global(locale("Russian_Russia.1251"));
	//объявление переменных
	int S,a,b;

	//вывод сообщения на консоль
	cout << "Введите ширину: ";
	//ввод значения переменной «a»
	cin >> a;
	//вывод сообщения на консоль
	cout << "Введите длину: ";
	//ввод значения переменной «b»
	cin >> b;
	//расчет площади
	S = a*b;
	//вывод площади на экран
	cout<<"Площадь = "<<S<<"\n";
	return 0;
}
```

Результат выполнения программы представлен ниже на рисунке 5.

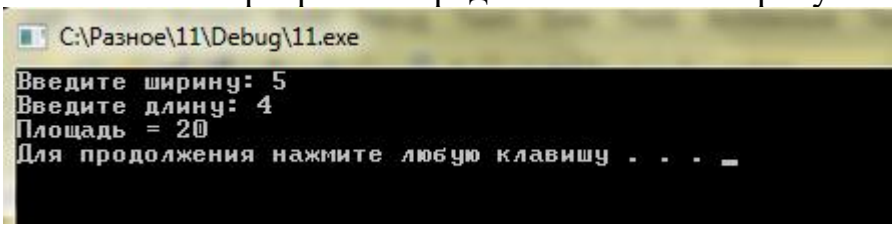


Рисунок 5 – Работа программы счета площади прямоугольника

Потоки ввода/вывода предусматривают возможность управления вводом-выводом на основе форматирующих функций-членов, флагов и манипуляторов. Флаги, функции и манипуляторы выполняют одну и ту же задачу – задают определенный формат ввода/вывода информации в потоках. Различие между ними состоит в способе их применения. Некоторые примеры форматирующих функций приведены в таблице 4.

Таблица 4 – Примеры форматирующих вывод функций

Пример функции	Описание
cout.fill(c);	Устанавливает символ заполнитель.
cout.width(w);	Задает ширину следующего выводимого поля в символах.
cout.precision(p);	Задает количество знаков после десятичной точки для следующего выводимого поля.

Для включения и выключения параметров ввода/вывода используются флаги форматирования. Чтобы установить флаг ввода/вывода, необходимо вызвать функцию «setf», если необходимо отключить флаг вывода, то используется функция «unsetf». Если при вводе/выводе необходимо установить или снять несколько флагов, то можно воспользоваться поразрядной логической операцией ИЛИ («|»). В этом случае конструкция языка C++ будет такой:

```
// установка нескольких флагов
cout.setf( ios::/*name_flag1*/ |…|ios::/*name_flag_n*/ );
// снятие нескольких флагов
cout.unsetf(ios::/*name_flag1*/|…|ios::/*name_flag_n*/ );
```

Некоторые примеры флагов форматирования приведены в таблице 5.

Таблица 5 – Примеры флагов форматирования

Пример флага	Описание
cout.setf(ios::oct);	Ввод/вывод величин в восьмеричной системе счисления.
cout.setf(ios::scientific);	Вывод чисел с плавающей точкой в экспоненциальной форме.
cout.setf(ios::fixed);	Вывод чисел с плавающей точкой в фиксированной форме (по умолчанию).

Следующий способ форматирования основывается на использовании манипуляторов, основные из которых приведены в таблице 6.

Таблица 6 – Примеры форматирующих манипуляторов

Пример манипулятора	Описание
cout<<endl;	Переход на новую строку при выводе.
int value = 64; cout << oct << value;	Вывод значения переменной в восьмеричной системе счисления.
double value = 1024.165; cout << scientific <<value;	Вывод чисел с плавающей точкой в экспоненциальной форме
double value = 1024.165; cout << fixed <<value;	Вывод чисел с плавающей точкой в фиксированной форме (по умолчанию).

double x = 13.5/2; cout<<setprecision(3) << x;	Задает количество знаков после запятой.

Для вывода символов в кодировке Unicode вместо потока вывода «cout» используют поток «wcout»:

wchar_t wc = L'q';

wcout << wc << endl;

1.3 Основные алгоритмические конструкции языка C++

1.3.1 Ветвление

В языке программирования C++ для организации ветвления (выбора операции на основе условия) используются операторы if, else и switch. Эти конструкции позволяют выбирать, какую операцию выполнить, в зависимости от условия, и выбирать между несколькими альтернативами.

```
if (condition1) {
    // случай, когда condition1 истинно
} else if (condition2) {
    // случай, когда condition1 ложно, а condition2 истинно
} else if (condition3) {
    // случай, когда condition1 и condition2 ложны, а condition3 истинно
} else {
    // случай, когда condition1, condition2 и condition3 ложны
}
```

Листинг 11 – Код программы на ветвление

```
#include <iostream>
int main() {
    int x;
    std::cin >> x;
    if (x <= 0) {
        std::cout << "zero or negative\n";
    } else if (x == 1) {
        std::cout << "one\n";
    } else if (x == 2) {
        std::cout << "two\n";
    } else {
        std::cout << "many\n";
    }
}
```

Оператор switch следует рассматривать как условный прыжок на соответствующую метку в зависимости от значения выражения.

Листинг 12 – Код программы на оператор switch

```
#include <cstdint>
#include <iostream>

int main() {
    int64_t a, b;
    char operation;
    std::cin >> a >> operation >> b;

    int64_t result;
    switch (operation) {
        case '+':
            result = a + b;
            break;  // если не написать этот break, программа просто пойдёт дальше в код следующего блока case
        case '-':
            result = a - b;
            break;
        case '*':
            result = a * b;
            break;
        case '/':
        case ':':
            result = a / b;
            break;
        case '%':
            result = a % b;
            break;
        default:  // здесь обрабатывается случай, когда ни один case не сработал.
            result = 0;
    }

    std::cout << result << "\n";
}
```

1.3.2 Циклы

Циклом называется блок кода, который для решения задачи требуется повторить несколько раз. Каждый цикл состоит из:

–блока проверки условия повторения цикла;

–тела цикла.

Цикл выполняется до тех пор, пока блок проверки условия возвращает истинное значение. Тело цикла содержит последовательность операций, которая выполняется в случае истинного условия повторения цикла. После выполнения последней операции тела цикла снова выполняется операция проверки условия повторения цикла. Если это условие не выполняется, то будет выполнена операция, стоящая непосредственно после цикла в коде программы.

В языке C++ используются три типа циклов:

Цикл с предусловием while - повторяет блок кода до тех пор, пока заданное условие истинно. Проверка условия происходит перед каждой итерацией.

Листинг 13 – Общая форма записи для цикла с предусловием while

```
while (Условие)
    {
      БлокОпераций;
    }
```

Листинг 14 – Код программы для цикла с предусловием while (вычисление суммы чисел с использованием цикла while)

```
#include <iostream>
using namespace std;

int main() {
    int sum = 0; // Инициализация переменной для хранения суммы
    int i = 1;   // Инициализация счетчика

    // Цикл будет выполняться, пока i меньше или равно 10
    while (i <= 10) {
        sum += i; // Добавить текущее значение i к sum
        i++;      // Увеличить счетчик на 1
    }

    cout << "Сумма чисел от 1 до 10 равна: " << sum << endl;
    // Вывод: Сумма чисел от 1 до 10 равна: 55
    return 0;
}
```

Цикл с постусловием do...while - это цикл с постусловием, где истинность выражения, проверяющего Условие проверяется после выполнения БлокаОпераций, заключенного в фигурные скобки. Тело цикла выполняется до тех пор, пока выражение, проверяющее Условие, не станет ложным, то есть тело цикла с постусловием выполнится хотя бы один раз.

Листинг 15 – Общая форма записи для цикла с постусловием do...while

```
do {
      БлокОпераций;
    } while (Условие);
```

Листинг 16 – Код программы для цикла с постусловием do...while (запрос на ввод данных до тех пор, пока не будет введено корректное значение с использованием do...while)

```
#include <iostream>
using namespace std;

int main() {
    int number;

    do {
        cout << "Пожалуйста, введите положительное число: ";
        cin >> number;
    } while (number <= 0); // Проверка условия после попытки ввода

    cout << "Вы ввели: " << number << endl;
    return 0;
}
```

Цикл for - параметрический цикл (цикл с фиксированным числом повторений).

Листинг 17 – Общая форма записи для цикла for

```
for (Инициализация; Условие; Модификация)
{
  БлокОпераций;
}
```

Листинг 18 – Код программы для цикла for (вывод последовательности чисел и вычисление факториала с использованием цикла for)

```
#include <iostream>
using namespace std;

int main() {
    int n;
    long long factorial = 1; // Используем long long для больших
чисел

    cout << "Введите положительное целое число: ";
    cin >> n;

    // for (инициализация; условие; изменение) { тело_цикла }
```

```
        for (int i = 1; i <= n; ++i) {
            factorial *= i; // Умножаем factorial на i на каждой
итерации
        }

        cout << "Факториал числа " << n << " = " << factorial << endl;
        // Для n = 5 вывод: Факториал числа 5 = 120
        return 0;
    }
```

1.3.3 Статические массивы

Массив - это непрерывный участок памяти, содержащий последовательность объектов одинакового типа, обозначаемый одним именем.

Массив характеризуется следующими основными понятиями:

–элемент массива (значение элемента массива) – значение, хранящееся в определенной ячейке памяти, расположенной в пределах массива, а также адрес этой ячейки памяти. Каждый элемент массива характеризуется тремя величинами: адресом элемента (адресом начальной ячейки памяти, в которой расположен этот элемент); индексом элемента (порядковым номером элемента в массиве); значением элемента;

–адрес массива – адрес начального элемента массива;

–имя массива – идентификатор, используемый для обращения к элементам массива;

–размер массива – количество элементов массива;

–размер элемента – количество байт, занимаемых одним элементом массива.

–длина массива – количество байт, отводимое в памяти для хранения всех элементов массива.

Для объявления массива в языке C++ используется следующий синтаксис:
тип имя[размерность]={инициализация};

Листинг 19 – Объявление статического массива

```
// Одномерный массив из N элементов типа Type
Type array_name[N];

// Двумерный массив из R строк и C столбцов
Type array_name[R][C];
```

где N, R, C — целочисленные константы, известные на этапе компиляции.

Инициализация представляет собой набор начальных значений элементов массива, указанных в фигурных скобках, и разделенных запятыми.

Если количество инициализирующих значений, указанных в фигурных скобках, меньше, чем количество элементов массива, указанное в квадратных скобках, то все оставшиеся элементы в массиве (для которых не хватило инициализирующих значений) будут равны нулю. Это свойство удобно использовать для задания нулевых значений всем элементам массива.

Если массив проинициализирован при объявлении, то константные начальные значения его элементов указываются через запятую в фигурных скобках. В этом случае количество элементов в квадратных скобках может быть опущено.

При обращении к элементам массива индекс требуемого элемента указывается в квадратных скобках [].

Одномерный массив можно представить как строку (вектор), где каждый элемент имеет свой порядковый номер (индекс).

Листинг 20 – Одномерный массив (ввод, обработка, вывод)

```
#include <iostream>
using namespace std;

int main() {
    const int size = 5;     // Размер массива – константа
    double numbers[size]; // Объявление массива из 5 вещественных
чисел
    double sum = 0.0;

    // Ввод данных в массив
    cout << "Введите " << size << " чисел: " << endl;
    for (int i = 0; i < size; ++i) {
        cout << "numbers[" << i << "] = ";
        cin >> numbers[i];
    }

    // Вычисление суммы элементов
    for (int i = 0; i < size; ++i) {
        sum += numbers[i];
    }

    // Вывод содержимого массива и результата
    cout << "Массив: [ ";
    for (int i = 0; i < size; ++i) {
        cout << numbers[i] << " ";
```

```
    }
    cout << "]" << endl;

    cout << "Среднее арифметическое: " << sum / size << endl;
    return 0;
}
```

Двумерный массив можно представить как таблицу (матрицу), состоящую из строк и столбцов. Для доступа к элементу требуется указать два индекса: сначала номер строки, затем номер столбца. Отличие многомерного массива от одномерного состоит в том, что в одномерном массиве положение элемента определяется одним индексом, а в многомерном — несколькими.

Листинг 21 – Двумерный массив (заполнение и вывод матрицы)

```
#include <iostream>
#include <iomanip> // Для setw()
using namespace std;

int main() {
    const int rows = 3, cols = 4;
    int matrix[rows][cols]; // Объявление матрицы 3x4

    // Заполнение матрицы последовательными числами
    int counter = 1;
    for (int i = 0; i < rows; ++i) {     // Цикл по строкам
        for (int j = 0; j < cols; ++j) { // Цикл по столбцам
            matrix[i][j] = counter++;
        }
    }

    // Вывод матрицы на экран
    cout << "Содержимое матрицы " << rows << "x" << cols << ":" <<
endl;
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            cout << setw(3) << matrix[i][j] << " "; // setw(3)
задает ширину вывода
        }
        cout << endl; // Перенос строки после вывода всех элементов
одной строки
    }
    // Вывод:
    // Содержимое матрицы 3x4:
    //   1   2   3   4
```

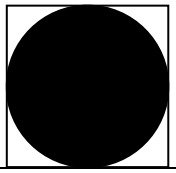
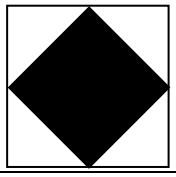
```
    //   5   6   7   8
    //   9  10  11  12
    return 0;
}
```

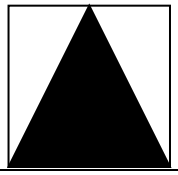
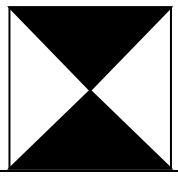
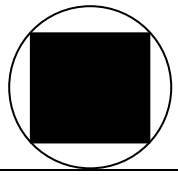
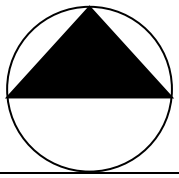
2 Задание к лабораторной работе

Задание 1

Создайте проект консольного приложения и напишите программу, вычисляющую значение функции в соответствии с вариантом задания (таблица 7). Значение параметров функции должны вводиться с клавиатуры, а вычисленное значение выводиться на экран.

Таблица 7. Варианты заданий

№ вар-та	Задание
1	Вычислить значение $z = x^3 + 2x^2 - 5y$ при заданных x и y.
2	Вычислить значение $z = x \cdot \sin y + \sqrt{\lvert xy \rvert}$ при заданных x и y.
3	Вычислить значение $z = \sqrt{\lvert x - \cos y \rvert}$ при заданных x и y.
4	Вычислить значение $z = -y^2 + 2x - 71xy$ при заданных x и y.
5	Вычислить значение $z = 2xy - \sin(x + 3y^2)$ при заданных x и y.
6	Вычислить значение $z = 2xy - \sqrt{\lvert \sin(x^2 + 2y^2) \rvert}$ при заданных x и y.
7	Вычислить значение $z = -x^y + y^x$ при заданных x и y.
8	Вычислить значение $z = (x + y)^2/(x + y)^3$ при заданных x и y.
9	Вычислить значение $z = \frac{e^x + e^y}{x+y}$ при заданных x и y.
10	Вычислить значение $z = \sin\left(\frac{\lvert x \rvert}{\lvert x \rvert + \lvert y \rvert}\right)$ при заданных x и y.
11	Вычислить значение $z = y^3 - 2x^2 + 7xy - 4$ при заданных x и y.
12	Вычислить значение $z = x^4 - y^2x^2 + 7x - 3$ при заданных x и y.
13	Вычислить значение $z = -x^y + y^x$ при заданных x и y.
14	Вычислить значение $z = \log_2\left(x + \frac{\sqrt{y}}{x^2}\right)$ при заданных x и y.
15	Вычислить значение $z = \log_{10} \frac{\sin x^2}{\lvert \cos y \rvert + 2}$ при заданных x и y.
16	Вычислить значение $x = -z^y + y^z$ при заданных z и y.
17	Вычислить площадь правильного пятиугольника как сумму площадей образующих его треугольников при длине стороны пятиугольника равной z.
18	 Вычислить площадь квадрата за вычетом площади вписанного в него круга. Сторону квадрата принять равной x.
19	 Вычислить площадь квадрата за вычетом площади вписанного в него меньшего квадрата, вершины которого расположены в центрах сторон большего квадрата. Сторону большего квадрата принять равной x.

№ вар-та	Задание
20	Вычислить площадь треугольника, если длины его сторон равны x, y и z. Считать, что треугольник с заданными сторонами существует.
21	 Вычислить площадь квадрата за вычетом площади вписанного в него треугольника, вершины которого расположены так, как указано на рисунке. Сторону квадрата принять равной x.
22	 Вычислить площадь квадрата за вычетом площади вписанных в него треугольников, вершины которых расположены так, как указано на рисунке. Сторону квадрата принять равной x.
23	 Вычислить площадь круга за вычетом площади вписанного в круг квадрата, как показано на рисунке. Радиус круга принять за x.
24	 Вычислить площадь круга за вычетом площади вписанного в него равнобедренного треугольника, как показано на рисунке. Радиус круга принять за x.
25	Вычислить выражение $y = (\cos z^3)^2 - \frac{z}{\sqrt{\lvert a^2+\tan z\rvert}}$ при заданных a и z.

Задание 2

Составить консольное приложение в соответствие с заданием. При реализации задания работаем только со статическими массивами. Варианты заданий приведены в таблице 8.

Таблица 8. Варианты заданий

№ варианта	Задание к лабораторной работе
1	1. Дан массив размера N и целое число K ($1 \le K \le N$). Перед элементом массива с порядковым номером K вставить новый элемент с нулевым значением. 2. Даны целые положительные числа M и N. Сформировать матрицу размера $M \times N$, у которой все элементы I-й строки имеют значение $10 \cdot I$ ($I = 1, \ldots, M$). Найти произведение элементов каждой строки и из значений сформировать одномерный массив.
2	1. Дан массив размера N и два целых числа K и M ($1 \le K \le N$, $1 \le M \le 10$). После элемента массива с номером K вставить M новых элементов с нулевыми значениями. 2. Даны целые положительные числа M и N. Сформировать матрицу размера $M \times N$, у которой все элементы J-го столбца имеют значение $5 \cdot J$

	($J = 1, \ldots, N$). Найти сумму элементов каждой строки и из значений сформировать одномерный массив.
3	1. Найти максимальное значение среди отрицательных элементов массива. 2. Даны целые положительные числа M, N и набор из M чисел. Сформировать матрицу размера $M \times N$, у которой в каждом столбце содержатся все числа из исходного набора (в том же порядке).
4	1. Дан массив размера N. Вставить элемент с нулевым значением перед минимальным и после максимального элемента массива. 2. Даны целые положительные числа M, N и набор из N чисел. Сформировать матрицу размера $M \times N$, у которой в каждой строке содержатся все числа из исходного набора (в том же порядке).
5	1. Дан массив A размера N и целое число K ($1 \le K \le$ N). Преобразовать массив, увеличив каждый его элемент на исходное значение элемента A_k. 2. Даны целые положительные числа M и N. Сформировать матрицу размера $M \times N$, у которой элементы заполняются по формуле $Z = \sum_{i=1}^{M} \sum_{j=1}^{N} \frac{j-1+1}{i+j}$. Найти сумму элементов каждой строки и из значений сформировать одномерный массив
6	1. Дано целое число N (> 1), а также первый член A и разность D *арифметической прогрессии*. Сформировать и вывести массив размера N, содержащий N первых членов данной прогрессии: $A, \quad A + D, \quad A + 2 \cdot D, \quad A + 3 \cdot D, \quad \ldots$. 2. Даны целые положительные числа M и N. Сформировать матрицу размера $M \times N$, у которой элементы заполняются по формуле $Z = \sum_{i=1}^{M} \sum_{j=1}^{N} \sin(i^3 + j^4)$. Найти сумму элементов каждой строки и из значений сформировать одномерный массив.
7	1. Даны целые числа N (> 2), A и B. Сформировать и вывести целочисленный массив размера N, первый элемент которого равен A, второй равен B, а каждый последующий элемент равен сумме всех предыдущих 2. Дана матрица размера $M \times N$. Для каждого столбца матрицы с четным номером (2, 4, …) найти сумму его элементов и результат записать в одномерный массив
8	1. Дано целое число N (> 1), а также первый член A и знаменатель D *геометрической прогрессии*. Сформировать и вывести массив размера N, содержащий N первых членов данной прогрессии: $A, \quad A \cdot D, \quad A \cdot D^2, \quad A \cdot D^3, \quad \ldots$. 2. Для массива a(к,к) найти наибольшее значение суммы между соседними элементами в строке. Результат записать в массив B(m,n) и вывести на экран
9	1. Дан целочисленный массив A размера 10. Вывести порядковый номер последнего из тех его элементов A_K, которые удовлетворяют двойному неравенству $A_1 < A_K < A_{10}$. Если таких элементов нет, то вывести 0.

	2. Найти сумму положительных элементов каждой строки матрицы X(M,N). Результат записать в массив B(n) и вывести на экран
10	1. Дан массив размера N и целые числа K и L ($1 \le K \le L \le$ N). Найти среднее арифметическое элементов массива с номерами от K до L включительно. 2. Заданы матрицы A(м,м) и B(м,м). Получить матрицу X(M, M), каждый элемент которой равен сумме соответствующих элементов матриц A и B. Результат записать в массив B(m,n) и вывести на экран.
11	1. Дан целочисленный массив размера *N*. Проверить, чередуются ли в нем четные и нечетные числа. Если чередуются, то вывести 0, если нет, то вывести порядковый номер первого элемента, нарушающего закономерность. 2. Вычислить суммы элементов каждого столбца матрицы X(N,N), и записать их в массив Y(N) и вывести на экран.
12	1. Найти количество отрицательных элементов массива. Если оно меньше половины количества элементов исходного значения, то заменить знак числа последнего элемента массива. 2. Дана квадратная матрица A порядка M. Найти сумму элементов каждой ее диагонали.
13	1. Дан массив размера *N*. Найти номера тех элементов массива, которые больше своего правого соседа, и количество таких элементов. 2. Дана квадратная матрица A порядка M. Найти среднее арифметическое элементов каждой ее диагонали
14	1. Дан массив размера *N*. Найти номер его первого локального минимума (*локальный минимум* — это элемент, который меньше любого из своих соседей). 2. Заданы матрицы A(м,м) и B(м,м). Получить матрицу X(M,M), каждый элемент которой равен сумме соответствующих элементов матриц A и B. Результат записать в массив B(m,n).
15	1. Дан массив размера *N*. Найти номер его последнего локального максимума (*локальный максимум* — это элемент, который больше любого из своих соседей). 2. Найти сумму положительных элементов каждой строки матрицы X(M,N). Результат записать в массив B(n) и вывести на экран.
16	1. Дан массив размера *N*. Найти номера тех элементов массива, которые больше своего левого соседа, и количество таких элементов. 2. Даны целые положительные числа *M* и *N*. Сформировать матрицу размера $M \times N$, у которой все элементы *I*-й строки имеют значение $10 \cdot I$ ($I = 1, \ldots, M$). Найти произведение элементов каждой строки и из значений сформировать одномерный массив.
17	1. Дано число *R* и массив *A* размера *N*. Найти элемент массива, который *наиболее близок* к числу *R* (то есть такой элемент A_K, для которого величина $\lvert A_K - R \rvert$ является минимальной). 2. Даны целые положительные числа *M* и *N*. Сформировать матрицу размера $M \times N$, у которой все элементы *J*-го столбца имеют значение $5 \cdot J$ ($J = 1, \ldots, N$). Найти сумму элементов каждой строки и из значений сформировать одномерный массив.

18	1. Дан массив A размера N. Сформировать новый массив B того же размера, элементы которого определяются следующим образом: $B_K = 2 \cdot A_K$, если $A_K < 5$, $A_K/2$ в противном случае. 2. Даны целые положительные числа M, N и набор из M чисел. Сформировать матрицу размера $M \times N$, у которой в каждом столбце содержатся все числа из исходного набора (в том же порядке).
19	1. Дан массив A размера N. Сформировать новый массив B того же размера по следующему правилу: элемент B_K равен среднему арифметическому элементов массива A с номерами от 1 до K. 2. Дана матрица размера $M \times N$. Для каждого столбца матрицы с четным номером (2, 4, …) найти сумму его элементов и результат записать в одномерный массив.
20	1. Дан массив размера N. Обнулить элементы массива, расположенные между его минимальным и максимальным элементами (не включая минимальный и максимальный элементы). 2. Для массива a(к,к) найти наибольшее значение суммы между соседними элементами в строке. Результат записать в массив B(m,n) и вывести на экран.
21	1. Дан массив A размера N. Найти минимальный элемент из его элементов с четными номерами: A_2, A_4, A_6, … 2. Даны целые положительные числа M и N. Сформировать матрицу размера $M \times N$, у которой все элементы I-й строки имеют значение $10 \cdot I$ ($I = 1, \ldots, M$). Найти произведение элементов каждой строки и из значений сформировать одномерный массив
22	1. Дан массив размера N и целое число K ($1 \le K \le N$). Перед элементом массива с порядковым номером K вставить новый элемент с нулевым значением. 2. Даны целые положительные числа M и N. Сформировать матрицу размера $M \times N$, у которой все элементы J-го столбца имеют значение $5 \cdot J$ ($J = 1, \ldots, N$). Найти сумму элементов каждой строки и из значений сформировать одномерный массив.
23	1. Дано число R и массив размера N. Найти два различных элемента массива, сумма которых наиболее близка к числу R, и вывести эти элементы 2. Даны целые положительные числа M, N и набор из M чисел. Сформировать матрицу размера $M \times N$, у которой в каждом столбце содержатся все числа из исходного набора (в том же порядке).
24	1. Найти количество четных элементов массива. Если оно больше половины количества элементов исходного значения, то ко всем четным значениям элементов прибавить 1. 2. Даны целые положительные числа M, N и набор из N чисел. Сформировать матрицу размера $M \times N$, у которой в каждой строке содержатся все числа из исходного набора (в том же порядке).
25	1. Дан массив размера N. Найти номер его последнего локального максимума (*локальный максимум* — это элемент, который больше любого из своих соседей).

	2. Даны целые положительные числа M и N. Сформировать матрицу размера $M \times N$, у которой элементы заполняются по формуле $Z = \sum_{i=1}^{M} \sum_{j=1}^{N} \frac{j-1+1}{i+j}$. Найти сумму элементов каждой строки и из значений сформировать одномерный массив.
26	1. Дан массив размера N. Найти номера тех элементов массива, которые больше своего правого соседа, и количество таких элементов. 2. Дана матрица размера $M \times N$. Для каждого столбца матрицы с четным номером (2, 4, …) найти сумму его элементов и результат записать в одномерный массив

3 Материально-техническое обеспечение работы

Аудитория для проведения лабораторных занятий должна быть укомплектована специализированной мебелью и индивидуальными компьютерами следующей минимальной комплектации:

- Процессор: не менее двух исполнительных ядер, совместимый с системой команд x86 и x64, с поддержкой аппаратной виртуализации.
- Оперативная память: не менее 8 Гб.
- Монитор: не менее 24" (дюймов) по диагонали.
- Наличие локальной сети со скоростью обмена не менее 1 Гб/сек.
- Наличие доступа в сеть Интернет со скоростью не менее 1 Мбит/сек.
- Наличие клавиатуры и манипулятора «мышь».

На компьютерах должно быть установлено следующее программное обеспечение:

- Операционная система: Microsoft Windows 10 (или выше) и/или одна из операционных систем семейства Linux, допускающих установку среды исполнения для C++. Допускается конфигурация, когда одна из операционных систем установлена внутри виртуальной машины (гипервизора).
- Среды разработки и исполнения программ: Microsoft Visual Studio 2022 Community Edition (или выше), Visual Studio Code, Dev-C++.

4 Порядок выполнения и сдачи работы

Для выполнения лабораторной работы рекомендуется придерживаться следующего порядка выполнения:

1. Ознакомится с темой и целями лабораторной работы
2. Изучить теоретический материал
3. Подготовить рабочее окружение

4. Разработать программный код лабораторной работы в соответствие с заданием
5. Произвести проверку работоспособности, выполнить тестирование и отладку кода
6. Проанализировать полученные результаты и примененные в ходе решения подходы
7. Выполнить самооценку и рефлексию
8. Сдать лабораторную работу преподавателю и получить от него обратную связь

При сдаче студентом лабораторной работы основным отчетом выступает исходный код самостоятельно созданной в процессе выполнения работы программы.

Исходный код должен быть отформатирован согласно принятым для используемого языка программирования стандартам. Является желательным наличие в исходном коде комментариев, описывающих основные части программы и особенности их функционирования. В то же время студент должен быть готов объяснить работу программы в целом и каждую отдельную ее часть при полном отсутствии комментариев (например, они могут быть удалены преподавателем при сдаче работы).

Обязательным условием сдачи является умение студента самостоятельно восстановить любой участок исходного кода программы (но не более 20 строк подряд) после его удаления. Удаленный участок должен быть самостоятельно набран студентом заново в присутствие преподавателя. При восстановлении участка запрещается использовать операции Undo (Отменить) текстовых редакторов или переписывание кода участка из других источников.

Исходный код сдаваемой программы должен быть представлен в электронном виде.

Сдача исходного кода ранее сдававшихся программ или программ, код которых выложен в сети Интернет, не допускается.

5 Контрольные вопросы к лабораторной работе

1. С помощью какой стандартной переменной можно ввести с клавиатуры значения переменных (при использовании библиотеки iostream)?
2. Что делает функция sqrt?

3. Какое пространство имен необходимо использовать для доступа к переменным cin и cout?
4. Чем отличаются операторы «=» и «==»?
5. В чем отличие операторов «+» и «+=»?
6. С какой функции начинается выполнение программы на C++?
7. Какую библиотеку (заголовочный файл) необходимо подключить к программе, чтобы иметь доступ к каналам ввода/вывода cin и cout?
8. Что такое выражение?
9. Что такое ассоциативность?
10. В чем различие префиксной и постфиксной операций инкремента и декремента?
11. Что является результатом логической операции? Какие существуют логические операции?
12. Какие существуют побитовые операции?
13. Что получается в результате применения операции битового сдвига?
14. Для чего используется оператор sizeof?
15. Какая запись правильная cout<<(x*=5); или cout<<x*=5; ?
16. Что будет выведено на экран в результате выполнения следующих действий cout<<(sizeof(int)>sizeof(long)); ?

6 Перечень использованных информационных ресурсов

1. Подбельский, В. В. Стандартный Си++ : учебное пособие / В. В. Подбельский. — Москва : Финансы и статистика, 2022. — 688 с. — ISBN 978-5-00184-081-7. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/309404 (дата обращения: 06.05.2024). — Режим доступа: для авториз. пользователей.

2. Рацеев, С. М. Программирование на языке Си / С. М. Рацеев. — 2-е изд., стер. — Санкт-Петербург : Лань, 2023. — 332 с. — ISBN 978-5-507-47236-9. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/351863 (дата обращения: 06.05.2024). — Режим доступа: для авториз. пользователей.

Редактор А.А. Литвинова

ЛР № 04779 от 18.05.01. В набор В печать

Объем 0,5 усл.п.л., уч.-изд.л. Офсет. Формат 60х84/16.
Бумага тип №3. Заказ № Тираж 75. Цена

Издательский центр ДГТУ
Адрес университета и полиграфического предприятия:
344010, г. Ростов-на-Дону, пл. Гагарина, 1.